

CT520 – Algoritmos e Estruturas de Dados

Curso EN-AEL – 5º Ano

2º Semestre

Docente: Prof. Cortez e Correia

cortez.correia@marinha.pt

Grafos.

Objetivos de aprendizagem

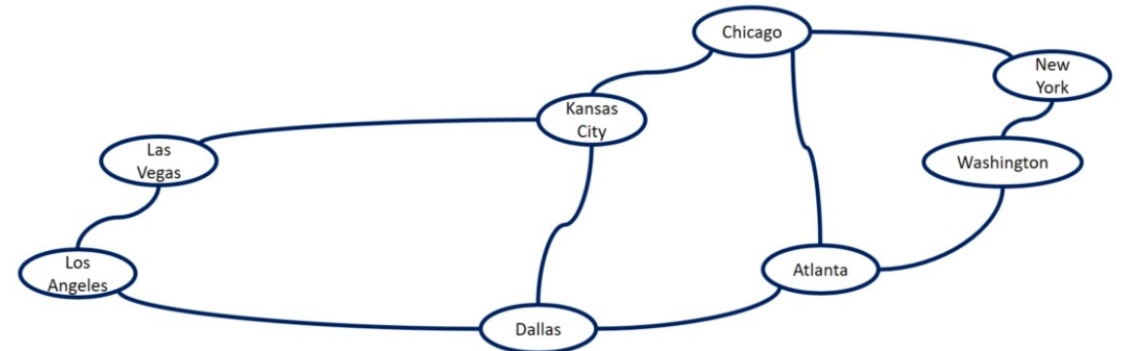
- Fornecer conhecimentos para compreender:
 - Estruturas de dados e algoritmos baseados em grafos

Sumário

- Terminologia sobre grafos
- Representação de grafos
- Procura em profundidade e em largura
- Árvore de abrangência mínima
- Caminho mais curto entre vértices

Grafos

- Um grafo (*graph*) representa uma coleção de objetos (vértices) e as ligações (arestas) entre pares desses objetos
- Os elementos de um grafo são:
 - um conjunto V de elementos Vértice/Nó (*Vertex/Node*)
 - um conjunto E de elementos Aresta/Arco (*Edge/Arc*)
 - uma função $w : E \rightarrow P(V)$ que associa a cada aresta um subconjunto de um ou dois elementos de V



Grafos

- Aplicações
 - Tráfego, transporte (controlo, gestão)
 - Navegação GPS
 - Abastecimento de água e redes de saneamento (gestão de carga)
 - Gestão de redes de energia
 - *Workflows* e cadeias de decisão
 - Planeamento e gestão de projetos
 - Compiladores, sistemas de ficheiros, jogos, criptografia, redes, Internet
 - Redes Bayesianas e probabilísticas
 - Robots (BFS)
 - Garbage Collector (BFS)
 - ...

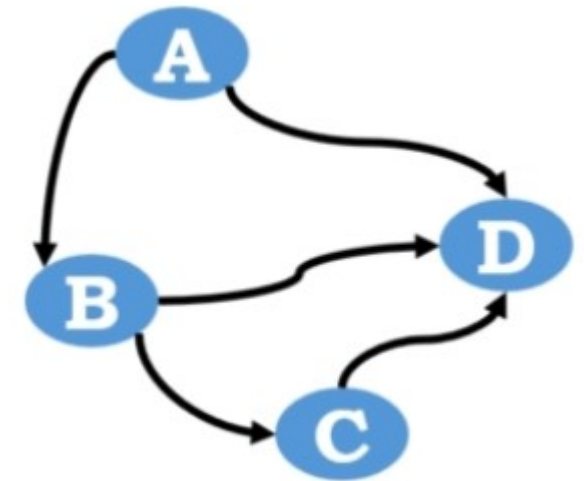
Grafos (*Graph*)

- Formalização: $\text{Graph } (G) = \{ V, E \}$

Vertices (V) = { A, B, C, D }

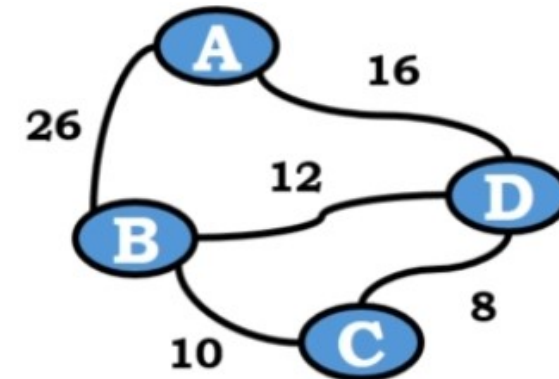
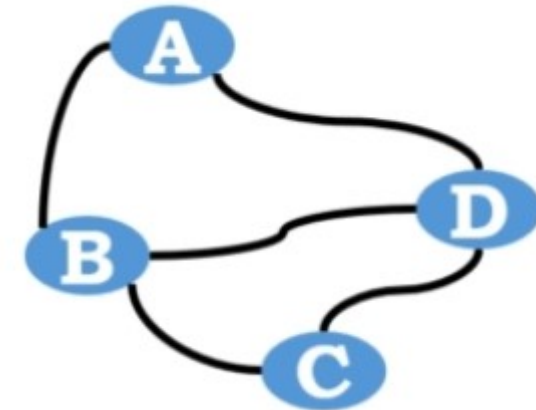
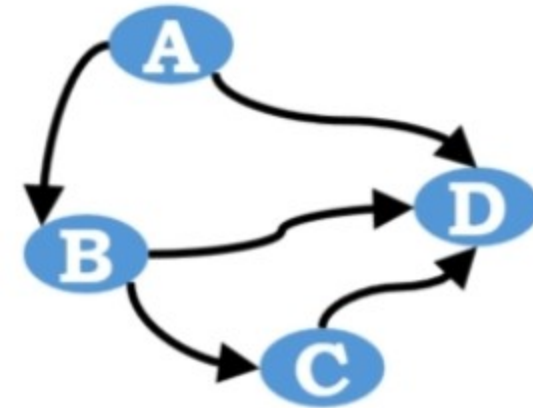
Edges (E) = { $A \rightarrow B$, $A \rightarrow D$, $B \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$ }

- uma **aresta** liga um par de vértices (v, w), em que $v, w \in V$
- um **vértice** w é adjacente a um vértice v se e só se $(v, w) \in E$
- uma aresta têm, por vezes, associada um **custo** ou **peso**



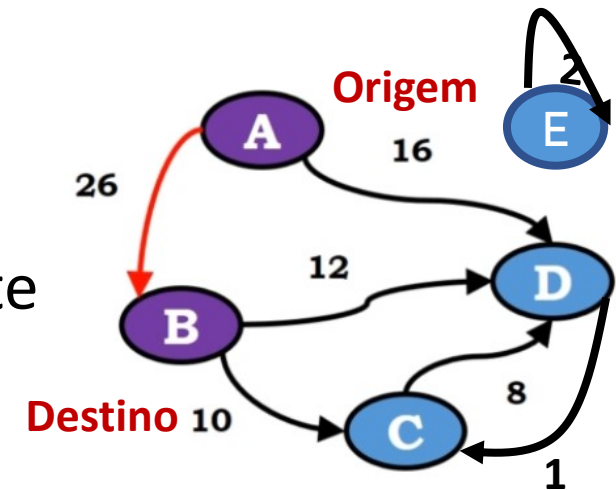
Grafos

- Tipos de grafos:
 - **Dirigido** ou **dígrafo** (*directed graph/digraph*): arestas orientadas
 - No caso da aresta $(v, w) \in E$, esta é dirigida (*directed edge*), se v preceder w
 - **Não-dirigido** (*undirected graph*): arestas sem direção
 - No caso da aresta $(v, w) \in E$, esta é não dirigida (*undirected edge*), não havendo precedência entre v e w
 - Havendo no grafo a aresta (v, w) e, daí (w, v) , w é adjacente a v e v é adjacente a w
 - **Misto** (*Mixed*): com arestas dirigidas e não-dirigidas
 - **Aresta pesada** (*weighted edge*): custo ou peso é atribuído a cada aresta (u, v)
 - Grafo pesado não-dirigido (*weighted undirected graph*)
 - Grafo pesado dirigido (*weighted directed graph*)



Grafos

- **Vértices finais** (*end vertices*): vértices terminais (origem/destino) de uma aresta
- **Vértices Adjacentes** (*adjacent vertices*): os vértices são adjacentes se existir uma aresta entre si
- **Aresta incidente** (*incident edge*) de/para um vértice, se este for um dos pontos de início/fim da aresta
 - Aresta incidente **do** Vértice A
 - Aresta incidente **para o** Vértice B
- Aresta dirigida:
 - **De saída** (*outgoing edge*): com origem no vértice
 - **De entrada** (*incoming edge*): com destino ao vértice
 - **Auto Ciclo** (*self loop*): se os dois pontos finais forem o mesmo
 - **Paralela** : aresta de u para v (u, v), havendo outra de v para u (v, u)



Grafos

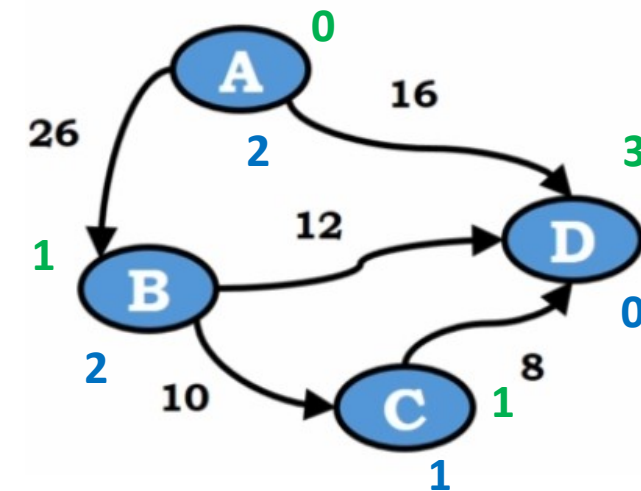
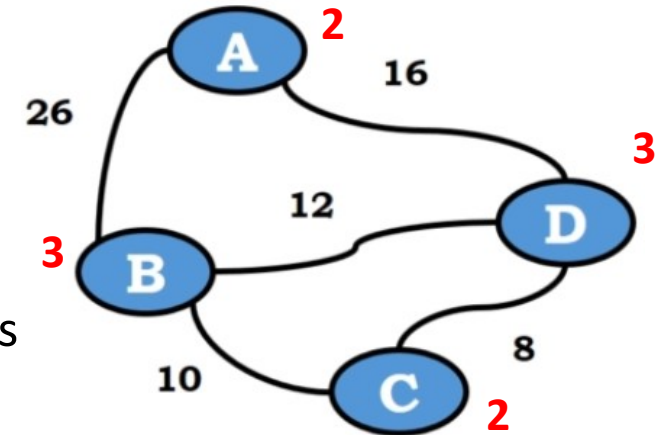
- Definições

- Grafo não dirigido:**

- Grau de um vértice** (*degree of Vertex*) - $\deg(v)$: número de arestas de/para o vértice

- Grafo dirigido:**

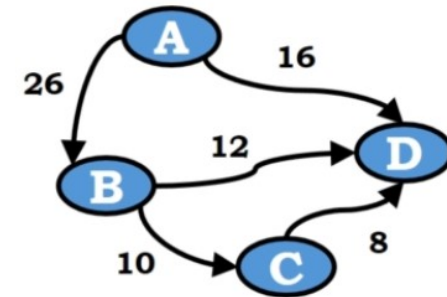
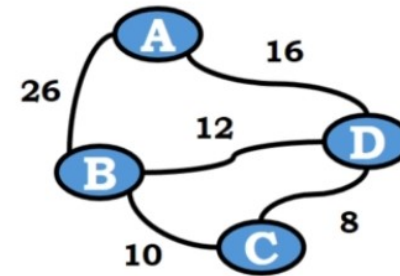
- Grau de entrada** (*in-degree*) - $\text{indeg}(v)$: número de arestas de entrada
 - Grau de saída** (*out-degree*) - $\text{outdeg}(v)$: número de arestas de saída



Grafos

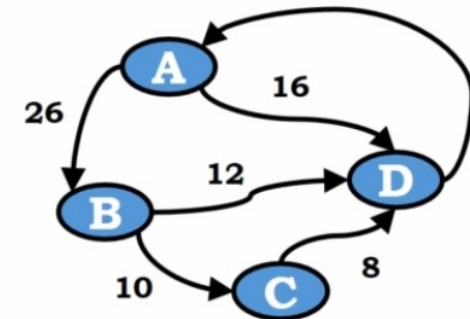
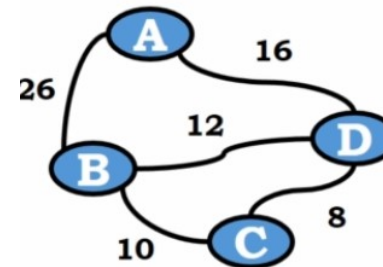
- **Caminho** (*path*): sequência de arestas a partir de um vértice terminando noutro vértice.

- Grafo não dirigido: ABC, ABCD, ABDC, ADBC,
- Grafo dirigido: ABC, ABCD, ABD, AD



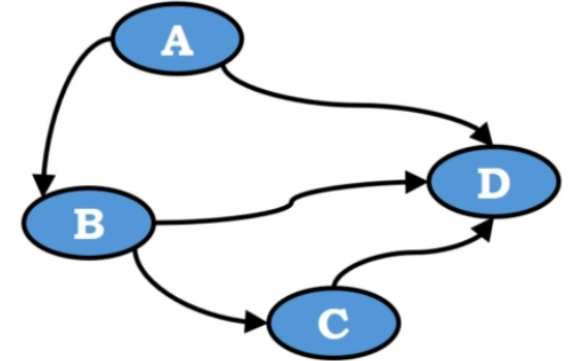
- **Ciclo**: caminho que começa e termina no mesmo vértice

- Grafo não dirigido: ABCDA, ABDA ...
- Grafo dirigido: ABCDA, ABDA , BCDAB



Grafos

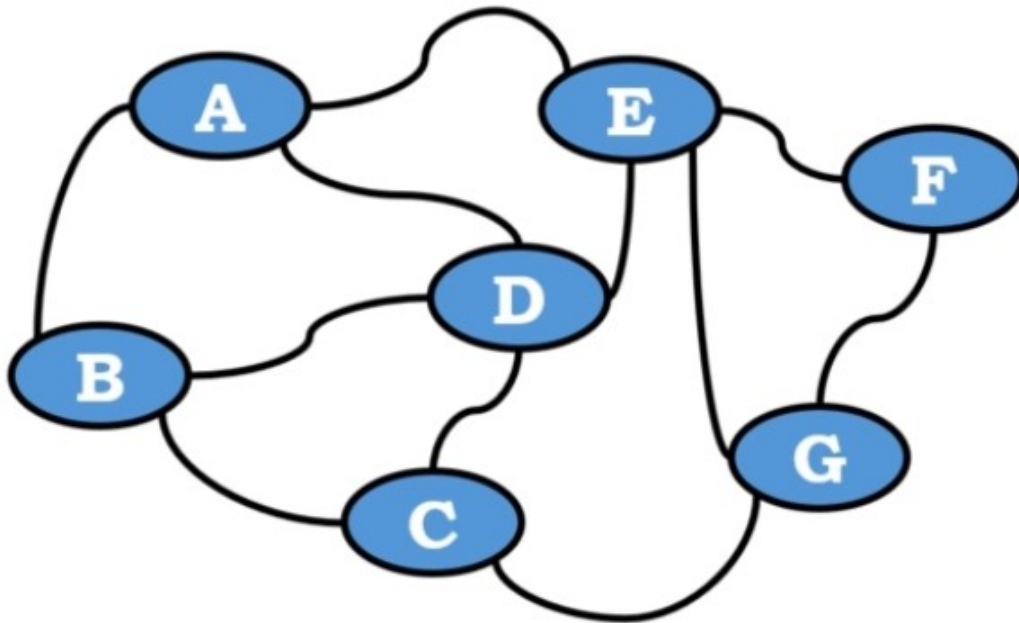
- **Grafo acíclico dirigido** (*Directed Acyclic Graph* - DAG): grafo dirigido sem ciclos



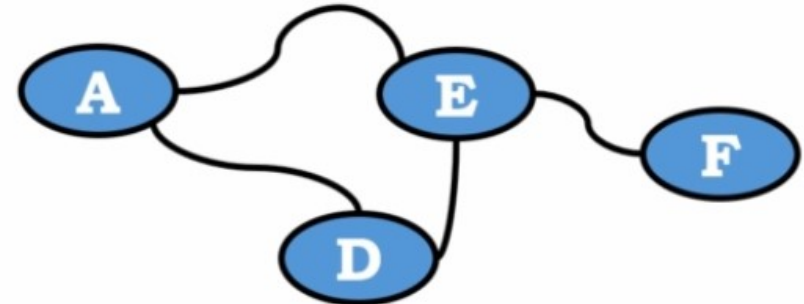
Grafos

- **Subgrafo** (*subgraph*): grafo cujos vértices e arestas são subconjuntos de vértices e arestas de outro grafo

G

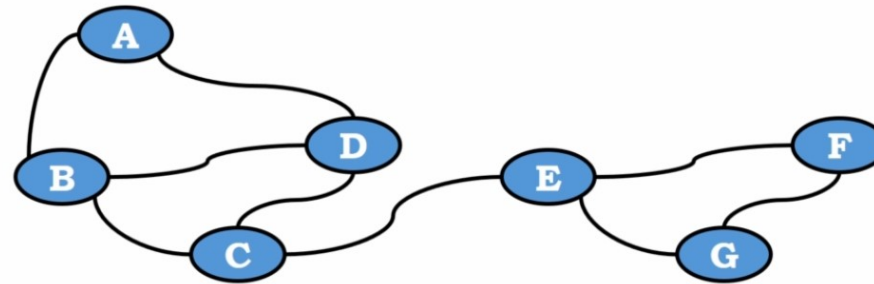


H

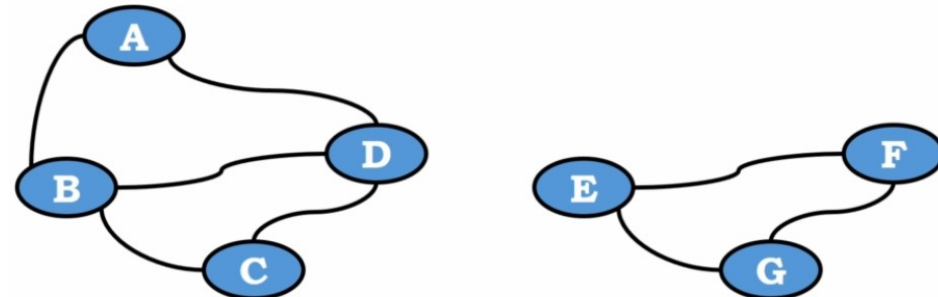


Grafos

- Grafo conectado: todos os vértices estão conectados por uma aresta

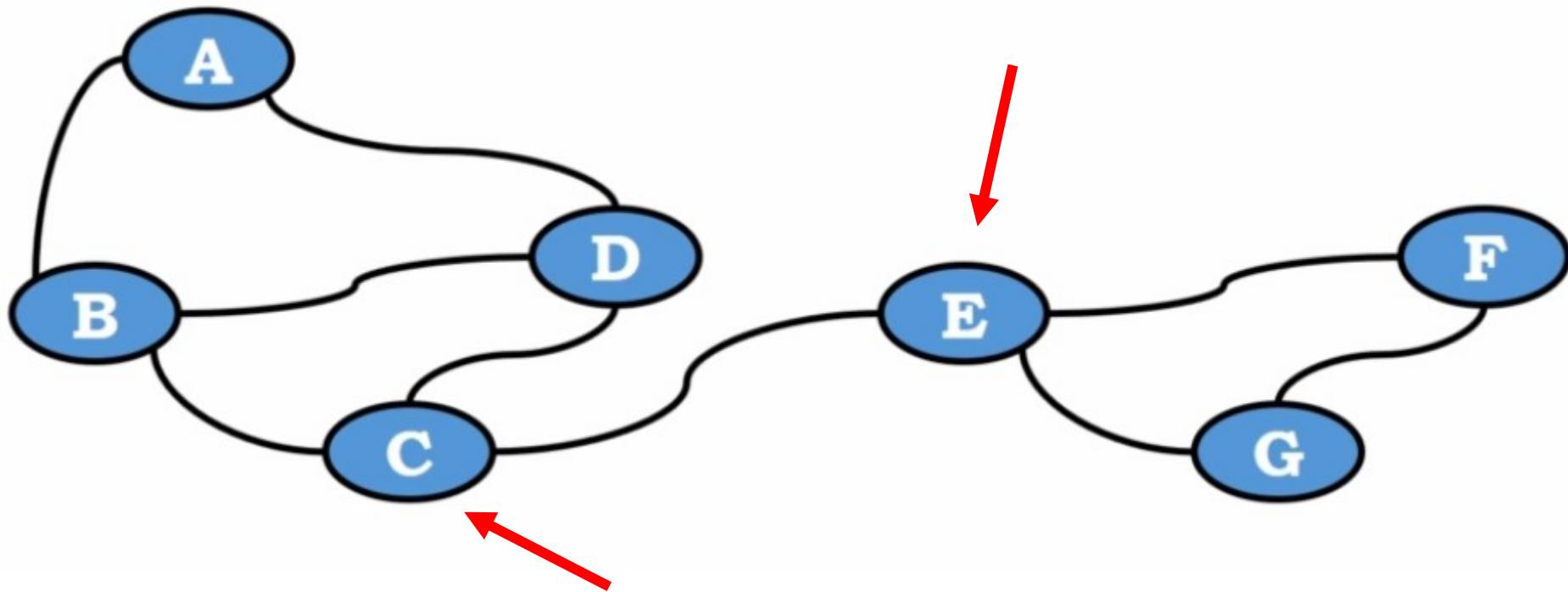


- **Componentes conectados** (*connected components*): subgrafos conectados que compõem o grafo não conectado



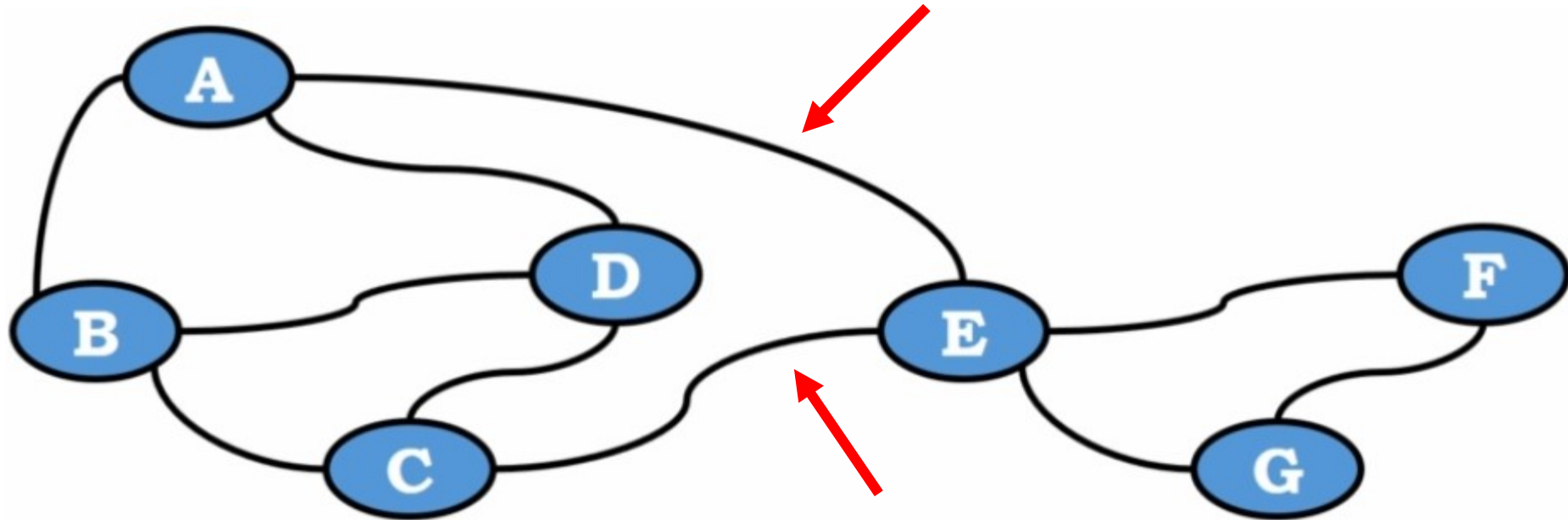
Grafos

- **Ponto de Articulação** (*articulation point*): vértice cuja remoção resulta em componentes conectados



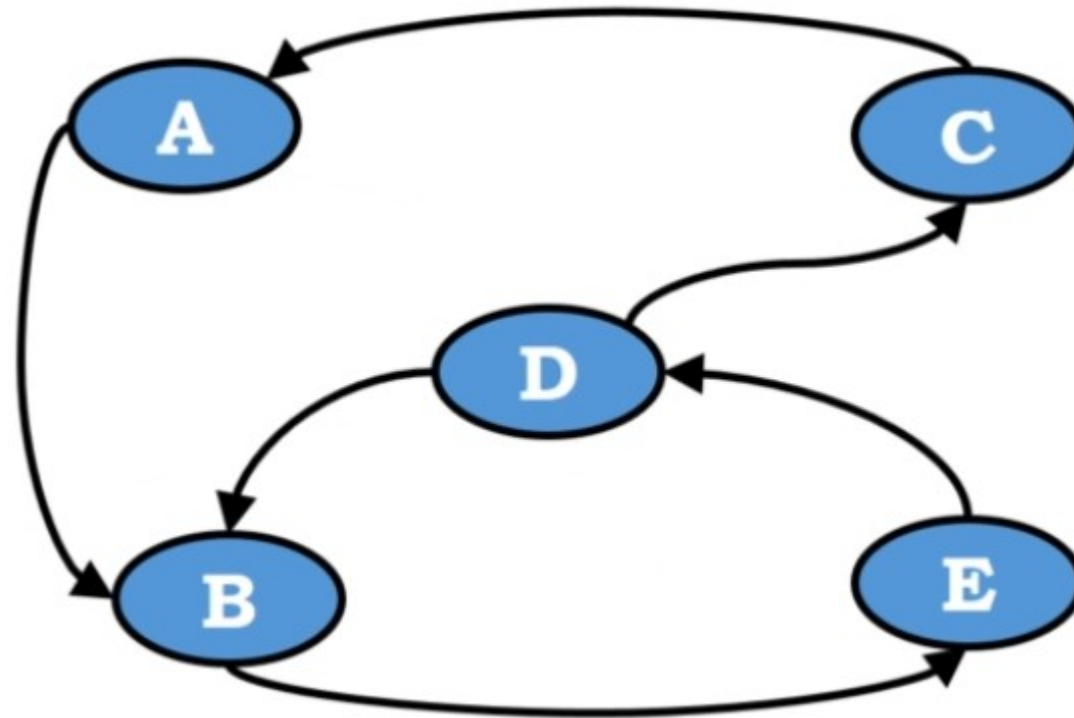
Grafos

- **Componentes bi-conectados** (*bi-connected components*): componentes conectados por duas arestas



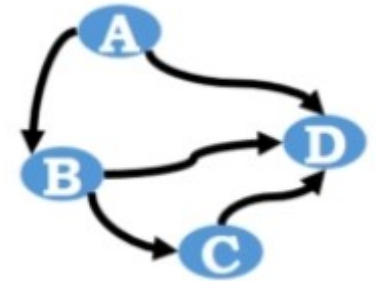
Grafos

- **Grafo fortemente conectado** (*Strongly Connected Graph*): todos os vértices são alcançáveis a partir de qualquer vértice



Grafos

- Grafo – Tipo Abstrato de Dados
 - Coleção de vértices e arestas
- Operações disponíveis:
 - **create(n)**: Cria um grafo com n vértices sem arestas
 - **Insert_edge(u, v, w = 1)**: cria uma aresta entre os vértices u a v, com peso w (por omissão 1)
 - **remove_edge(u, v)**: remove a aresta entre os vértices u para v
 - **exist_edge(u, v)**: retorna verdadeiro se a aresta entre os vértices u e v existe; caso contrário, retorna falso
 - **vertex_count()**: retorna o número de vértices no grafo
 - **edge_count()**: retorna o número de arestas no grafo
 - **vertices()**: retorna todos os vértices do grafo
 - **edges()**: retorna todas as arestas do grafo
 - **degree(u)**: retorna o grau do vértice u
 - **indegree(u)**: retorna o número de arestas que entram no vértice u
 - **outdegree(u)**: retorna o número de arestas que saem do vértice u

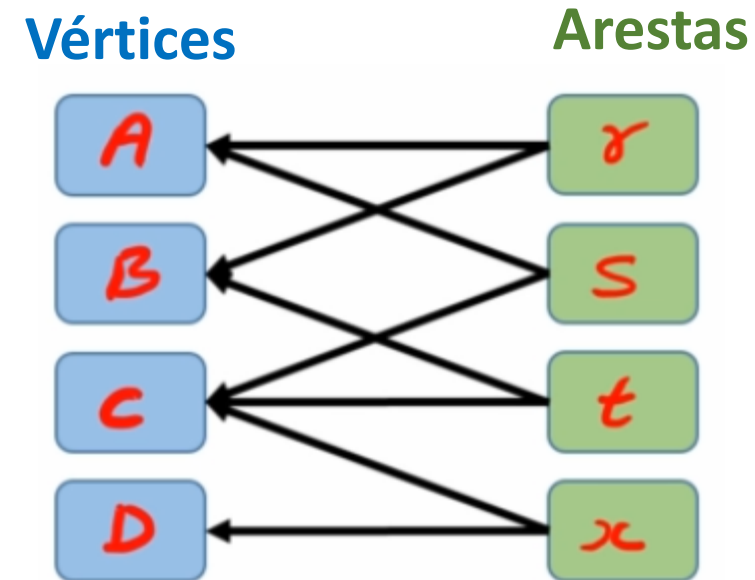
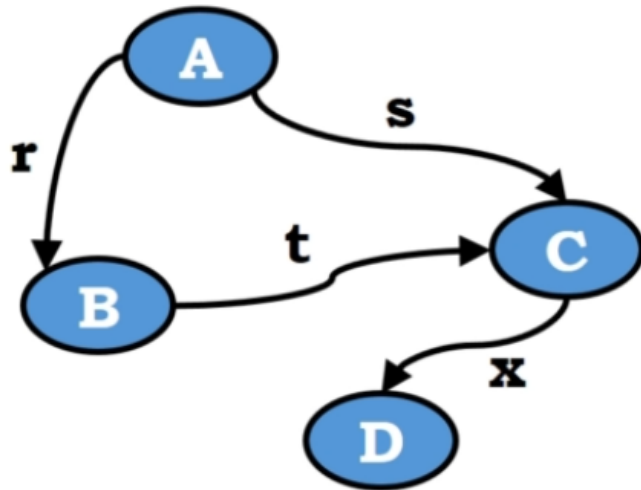


Grafos

- Representação de grafos: pode ser efetuada através de diferentes estruturas de dados alternativas:
 - **Lista de Arestas** (*Edge List*): mantém a lista de todas as arestas ligadas aos vértices
 - **Lista de Adjacências** (*Adjacency List*): para cada vértice é mantida uma lista separada das suas arestas incidentes
 - **Matriz de Adjacências** (*Adjacency Matrix*): mantida uma matriz quadrada de vértices, onde cada célula armazena uma referência à aresta entre os vértices correspondentes

Grafos

- Representação de grafos: **Lista de Arestas** (*Edge List*): mantém a lista de todas as arestas
 - Exemplo: Grafo pesado dirigido



Grafos

- Representação de grafos: **Lista de Arestas** (*Edge List*)

Operação **Complexidade
Temporal**

<code>insert_edge(u, v, w=1)</code>	$O(1)$
<code>remove_edge(u, v)</code>	$O(1)$
<code>exist_edge(u, v)</code>	$O(m)$
<code>vertex_count()</code>	$O(1)$
<code>edge_count()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$

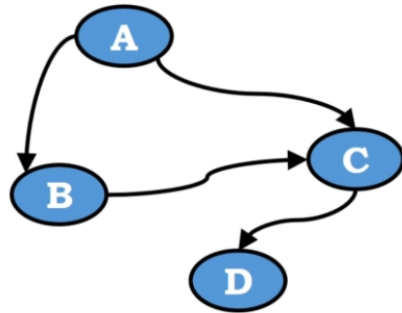
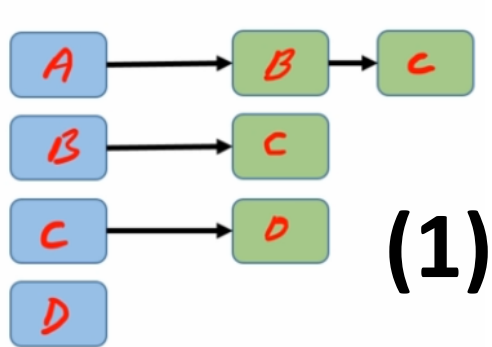
Vértices: n **Arestas: m**

**Complexidade Espacial
 $O(n+m)$**

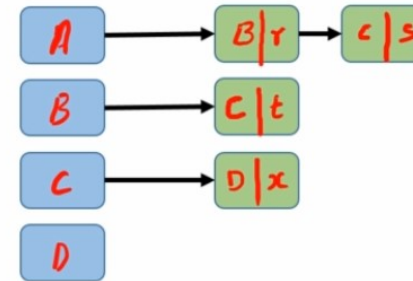
Grafos

- Representação de grafos: **Lista de Adjacências** (*Adjacency List*): para cada vértice é mantida uma lista separada das suas arestas incidentes
 - Exemplos: (1) Grafo dirigido; (2) Grafo pesado dirigido; (3) Grafo não-dirigido;

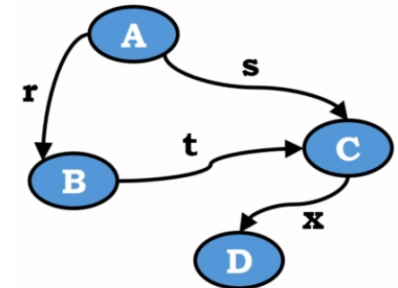
Vertices



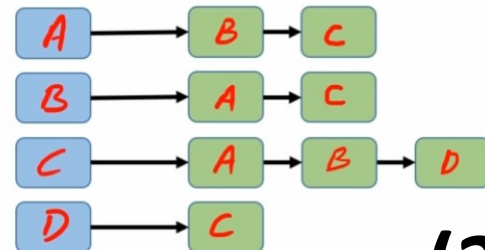
Vertices



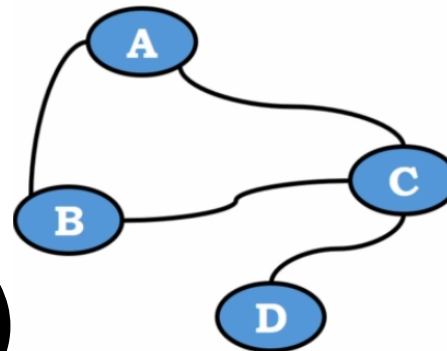
(2)



Vertices



(3)



Grafos

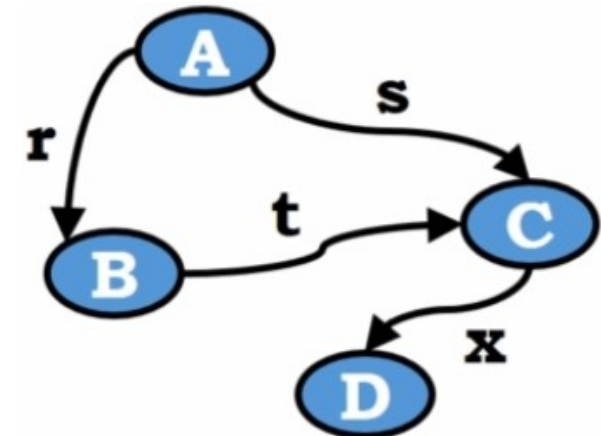
- Representação de grafos: **Lista de Adjacências** (*Adjacency List*)

Operação **Complexidade Temporal**

<code>insert_edge(u, v, w=1)</code>	$O(1)$
<code>remove_edge(u, v)</code>	$O(1)$
<code>exist_edge(u, v)</code>	$O(\min(d_u, d_v))$
<code>vertex_count()</code>	$O(1)$
<code>edge_count()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$

Vértices: n **Arestas:** m

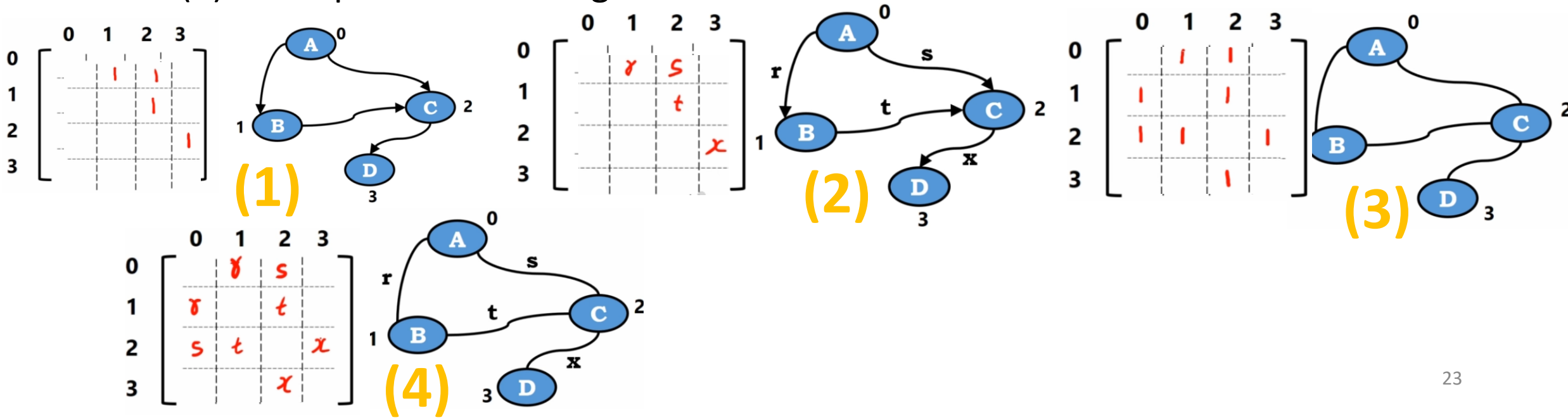
Complexidade Espacial
 $O(n+m)$



Grafos

$$A[i,j]=1 \text{ ou } 0$$
$$i \rightarrow u; j \rightarrow v$$

- Representação de grafos: **Matriz de Adjacências** (*Adjacency Matrix*): mantida uma matriz quadrada de vértices, onde cada célula armazena uma referência à aresta entre os vértices correspondentes
 - Exemplos: (1) Grafo dirigido; (2) Grafo pesado dirigido; (3) Grafo não-dirigido; (4) Grafo pesado não-dirigido



Grafos

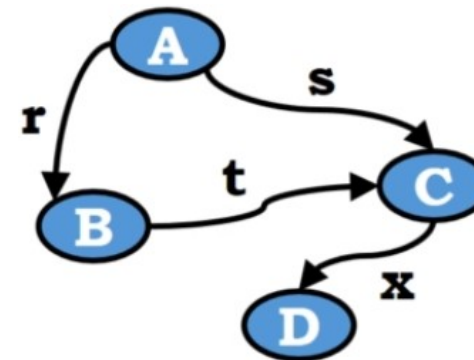
- Representação de grafos: **Matriz de Adjacências** (*Adjacency Matrix*)

Operação **Complexidade Temporal**

<code>insert_edge(u, v, w=1)</code>	$O(1)$
<code>remove_edge(u, v)</code>	$O(1)$
<code>exist_edge(u, v)</code>	$O(1)$
<code>vertex_count()</code>	$O(1)$
<code>edge_count()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>degree(u)</code>	$O(n)$

Vértices: n **Arestas:** m

Complexidade Espacial
 $O(n \times n) = O(n^2)$



Grafos

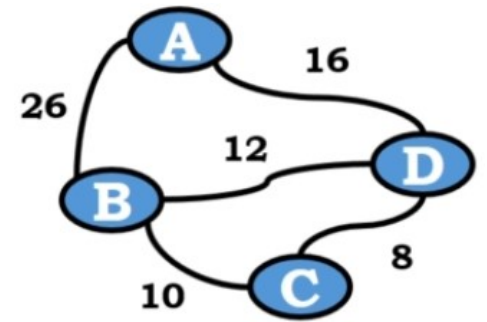
- Resumo da eficiência das 3 estruturas de dados

	Edge List	Adjacency List	Adjacency Matrix
Space Complexity	$O(n + m)$	$O(n + m)$	$O(n^2)$
Operation	Edge List	Adjacency List	Adjacency Matrix
insert_edge(u, v, w=1)	$O(1)$	$O(1)$	$O(1)$
remove_edge(u, v)	$O(1)$	$O(1)$	$O(1)$
exist_edge(u, v)	$O(m)$	$O(\min(d_u, d_v))$	$O(1)$
vertex_count()	$O(1)$	$O(1)$	$O(1)$
edge_count()	$O(1)$	$O(1)$	$O(1)$
vertices()	$O(n)$	$O(n)$	$O(n)$
edges()	$O(m)$	$O(m)$	$O(m)$
degree(u)	$O(m)$	$O(1)$	$O(n)$

Grafos

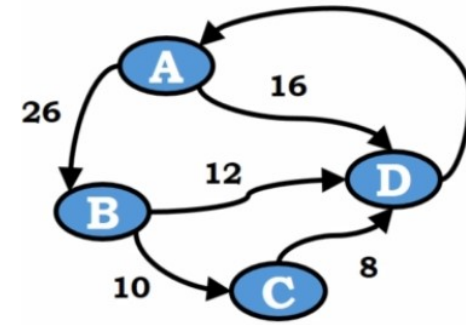
- Procedimentos em grafos:
 - **Procura** (*traversal*) é o procedimento sistemático de exploração do grafo
 - **Exploração**: consiste em examinar todos os vértices e arestas do grafo
 - **Eficiência** (*efficient traversal*): visita dos vértices e arestas em tempo linear do número destes
 - Grafo (estrutura não-linear)
 - Procura é diferente da usada em estruturas lineares (vetor, lista ligada)

Grafos



- Algoritmos de **procura** (*traversal*) para **grafos não-dirigidos**, pretendem resolver problemas relevantes, tais como:
 - Determinar acessibilidade (*reachability*) (se existe e qual o caminho) de um vértice para outro [DFS]
 - Determinar o melhor caminho, a partir de um vértice inicial, para um ou todos os restantes vértices [Bellman-Ford, Dijkstra]
 - Verificar se um grafo é conectado [DFS]
 - Determinar os componentes conectados do grafo [DFS]
 - Determinar ciclos no grafo [BFS, DFS]
 - Determinar uma árvore de abrangência mínima (*minimum spanning tree*) do grafo [Kruskall, Prim]
 - ...

Grafos



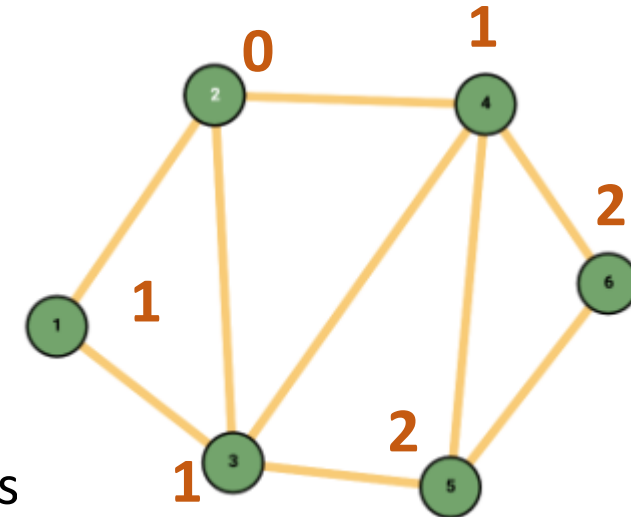
- Algoritmos de **procura** (*traversal*) para **grafos dirigidos**, pretendem resolver problemas relevantes, tais como:
 - Determinar o caminho mais curto de um vértice origem a outro vértice destino [Bellman-Ford, Dijkstra]
 - Determinar todos os vértices que podem ser alcançados a partir de um vértice origem [Bellman-Ford, Dijkstra]
 - Determinar se um grafo é fortemente conectado [DFS, BFS]
 - Determinar se um grafo é acíclico [DFS]
 -

Grafos

- Algoritmos básicos de **procura** (*traversal*) em grafos:
 - Procura em largura (*breadth-first search*)
 - Procura em profundidade (*depth-first search*)

Procura em Largura (*Breadth-First Search*)

- A BFS separa os vértices em diferentes níveis, visitando, em cada iteração, os vértices do nível, antes de passar, na iteração seguinte, aos vértices do próximo nível.
 - Inicia a procura no vértice inicial, considerado no nível 0
 - Identifica todos os vértices alcançáveis, a partir do vértice inicial, como sendo do nível 1, procede à sua visita e marcação como visitados
 - Na próxima iteração, identifica novos vértices alcançáveis a partir dos vértices de nível 1, que ainda não foram visitados, procede à sua visita e marcação como visitados
 - Este processo continua até que não haja mais vértices não-visitados
 - Complexidade temporal: $O(V+E)$; complexidade espacial: $O(V)$

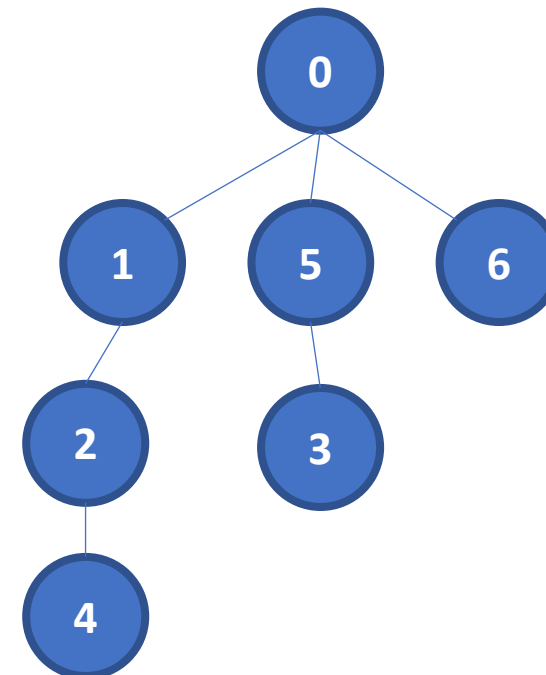
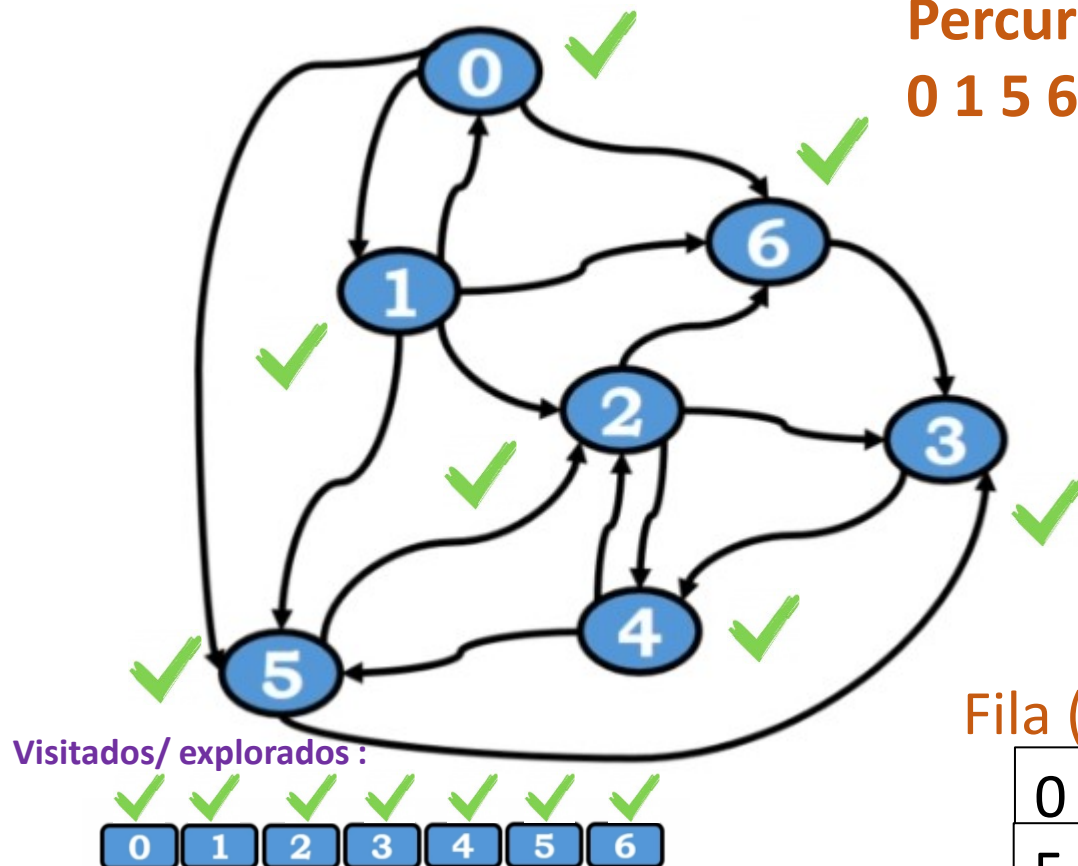


Procura em Largura (*Breadth-First Search*)

- **Exemplo 1** - BFS: nó inicial 0; visita aos nós por ordem ascendente

Percurso BFS: 0 1 5 6 2 3 4
Árvore de abrangência BFS

Nível



Fila (descobertos)

0	1 5 6	5 6 2 5 6	6 2 5 6 2 3	2 5 6 2 3 3
5 6 2 3 3 3 4	3 3 4 4	4		

0

1

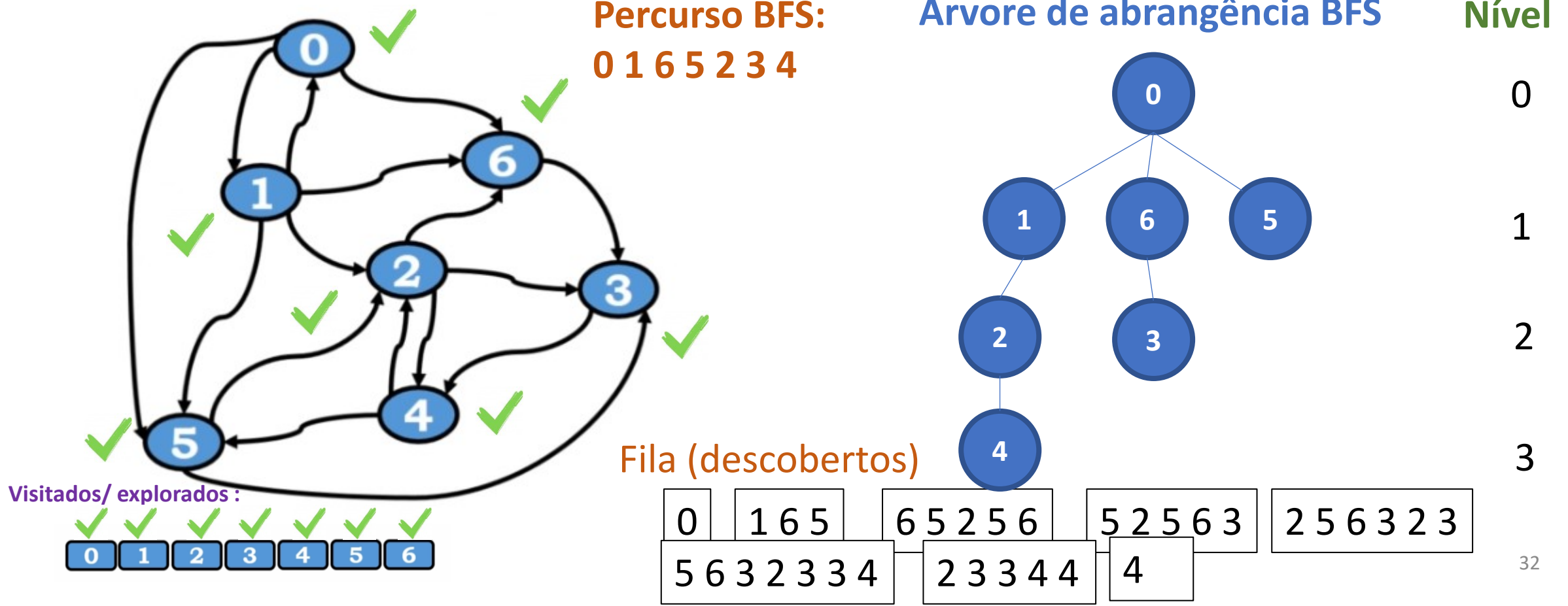
2

3

31

Procura em Largura (*Breadth-First Search*)

- **Exemplo 2** - BFS: nó inicial 0; visita aos nós por ordem aleatória



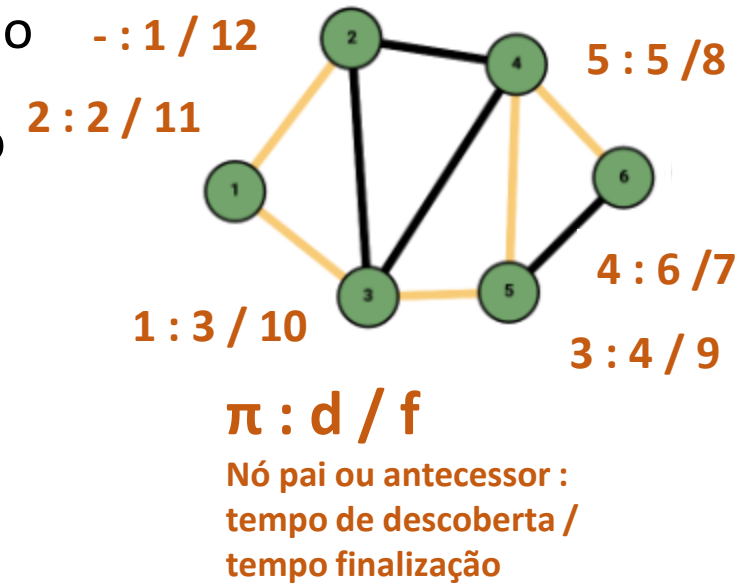
Procura em Largura (*Breadth-First Search*)

- BFS: Algoritmo

```
function BFS (s)
    i = s , q = Queue() , visited [] = [0,0,0.....n-1]
    print(i)
    visited[i] = 1
    q.enqueue(i)
    while ! q.isEmpty() then
        i = q.dequeue()
        for ( j = 0, j < n, j++)
            if adjmat[i][j]==1 && visited[j] == 0 then
                print(j)
                visited[j] = 1
                q.enqueue(j)
```

Procura em Profundidade (*Depth-First Search*)

- A DFS envolve os seguintes procedimentos:
 - Inicia-se num vértice que passa a ser o vértice corrente e é marcado como visitado
 - Prossegue seleccionando um vértice adjacente, acessível a partir do vértice inicial
 - O vértice adjacente é visitado e marcado como visitado (se não foi anteriormente visitado, razão para ser ignorado), tornando-se o vértice corrente
 - O procedimento anterior continua, até que não haja mais vértices adjacentes ao corrente vértice que não tenham sido explorados, fazendo *backtracking* e continuando nas arestas inexploradas. Finalmente alcança-se o vértice inicial, não havendo mais arestas inexploradas a partir do vértice inicial, terminando então a procura em profundidade
 - Complexidade temporal: $O(V+E)$; complexidade espacial: $O(V)$



Procura em Profundidade (*Depth-First Search*)

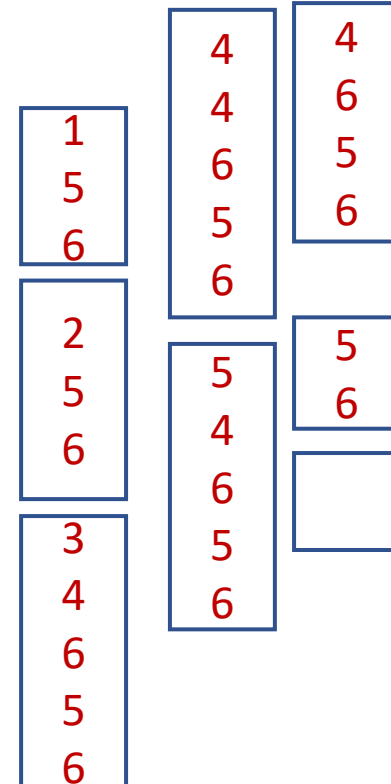
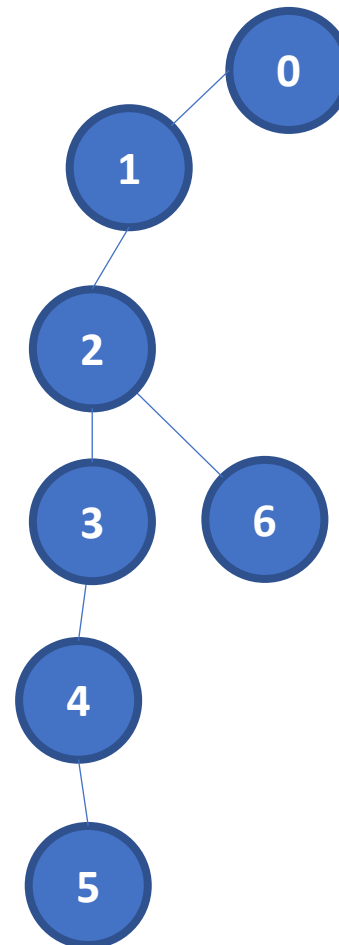
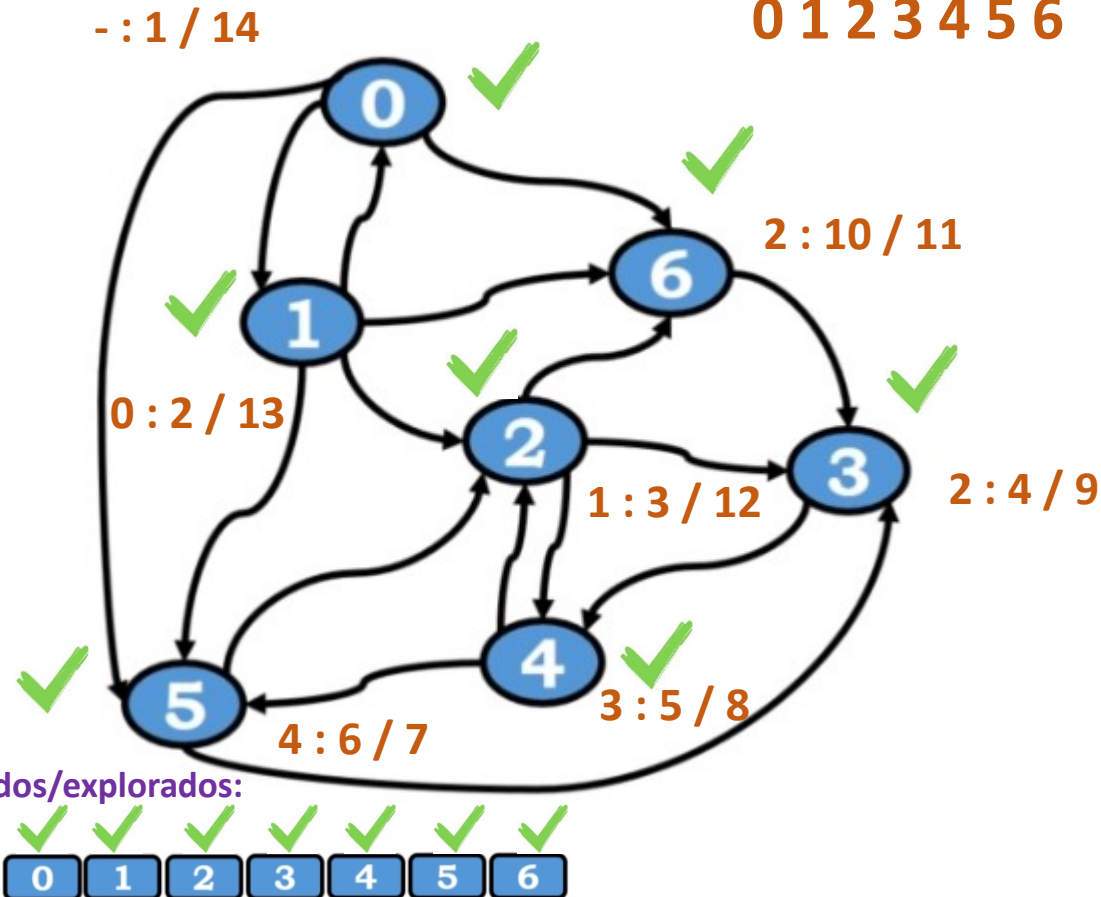
- **Exemplo 1** - DFS: nó inicial 0; visita aos nós adjacentes por ordem ascendente

Percurso DFS:

0 1 2 3 4 5 6

Árvore de abrangência DFS

Pilha (descobertos)



Procura em Profundidade (*Depth-First Search*)

- DFS: Algoritmo

```
function DFS (s)
    if visited[s] == 0 then
        print(s)
        visited[s] = 1
        for ( j = 0, j < n, j++)
            if adjmat[s][j]==1 && visited[j] == 0 then
                DFS(j)
```

BFS vs. DFS

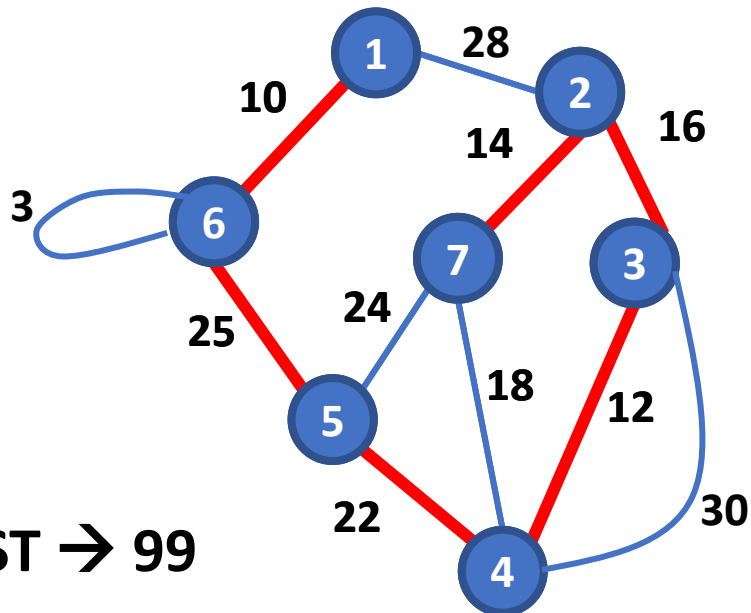
- BFS:
 - usado por robots para descoberta do ambiente; e algoritmos de *garbage collector* para descobrir referências ativas
 - mais exigente de recursos de memória (necessita de guardar referências dos nós vizinhos)
 - Preferível se o objetivo estiver perto da raiz
- DFS :
 - usado em árvores de decisão
 - Mais rápido e menos exigente em memória
 - Preferível se o objetivo estiver longe da raiz

Árvore de abrangência mínima

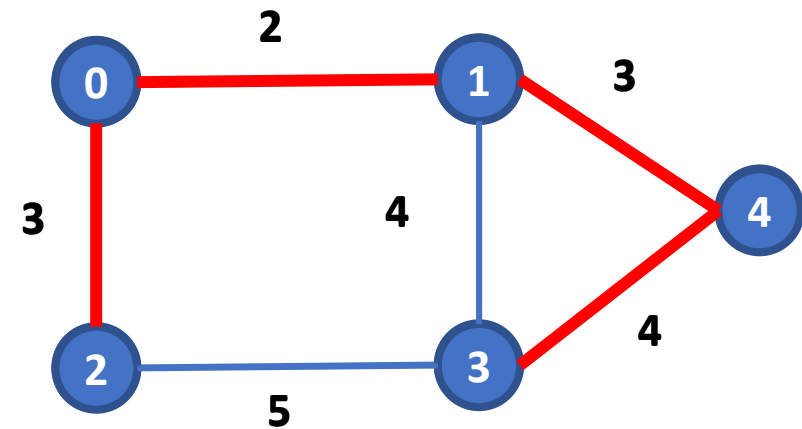
- Árvore de abrangência mínima (MST - *Minimum Spanning Tree*) – um subgrafo S de G , ligando todos os vértices de G com apenas $|V|-1$ arestas de E . Na MST não existem ciclos.
 - $S \subseteq G$, $S = (V', E')$ com $V' = V$ e $|E'| = |V|-1$
- Solução para a MST:
 - Algoritmo de *Prim*
 - Algoritmo de *Kruskal*
 - **Nota:** são ambos **algoritmos gulosos** (*Greedy Algorithms*) em que solução é construída componente a componente, com a decisão de escolher o próximo componente a ser efetuada com base no fato desse componente providenciar um benefício imediato. Adicionalmente, o algoritmo nunca põem em causa as escolhas efetuadas anteriormente.

Árvore de abrangência mínima

- Algoritmo de *Prim*
 - 1. Remover auto ciclos. Remover arestas paralelas de maior custo.
 - 2. Selecionar um vértice aleatório. Escolher a aresta de menor peso a partir desse vértice
 - 3. Acrescentar (ligar) à árvore anterior a aresta seguinte de menor peso
 - 4. Repetir o passo 2 até obter uma árvore com $|V|-1$ arestas
- Complexidade temporal: $O(|V| |E|) = O(n^2)$**



Custo da MST → 99



Custo da MST → 12

Árvore de abrangência mínima

- Algoritmo de *Kruskal* **Complexidade temporal: $O(|V| |E|) = O(n \log n)$**

1. Remover auto ciclos. Remover arestas paralelas de maior custo.
2. Selecionar a aresta de menor peso
3. Acrescentar a aresta de menor peso seguinte, sem que haja ciclos
4. Repetir o passo 2 até atingir o número de $|V|-1$ arestas

1-6 - 10

3-4 - 12

2-7 - 14

2-3 - 16

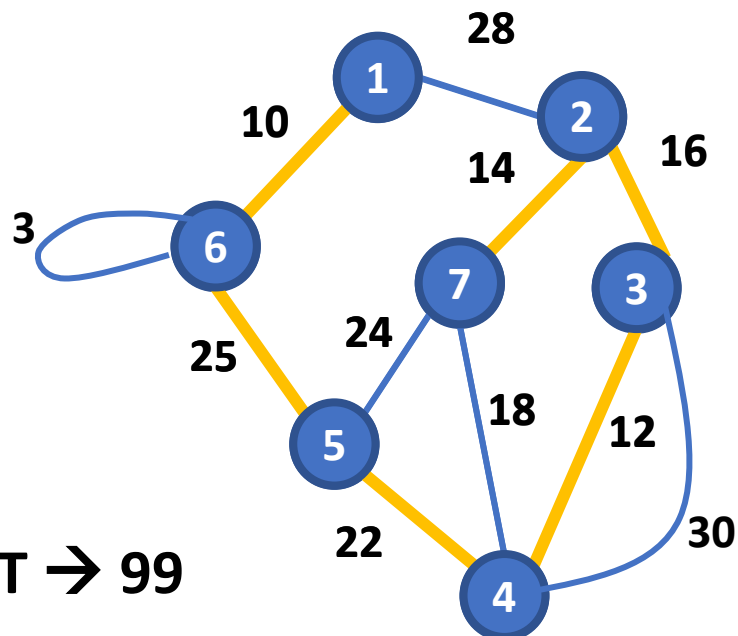
7-4 - 18

5-4 - 22

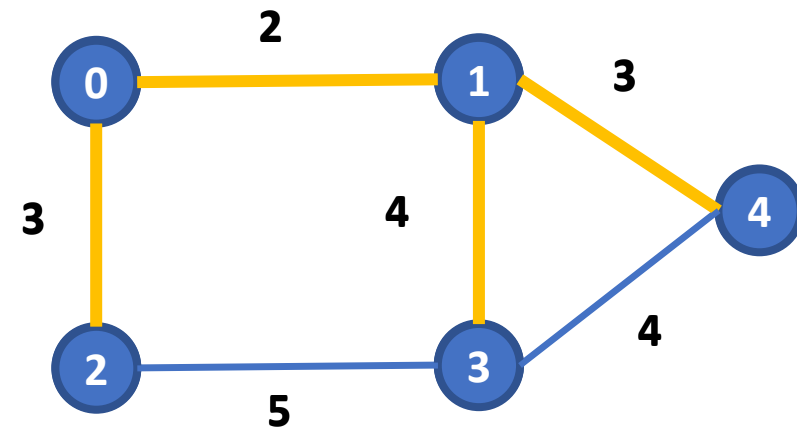
5-7 - 24

5-6 - 25

1-2 - 28



Custo da MST → 99



Custo da MST → 12

0-1 - 2

1-4 - 3

0-2 - 3

1-3 - 4

3-4 - 4

3-2 - 5

Caminho mais curto

Relaxamento (u,v)
if ($d[u] + w(u,v) < d[v]$)
 $d[v] = d[u] + w(u,v)$

- Caminho mais curto entre um vértice e todos os outros vértices do grafo (*single source shortest path*).
 - Algoritmo de *Dijkstra* (**guloso**): aplica-se a grafos sem arestas negativas
 - Executar o relaxamento das arestas de cada um dos $|V|$ vértices a visitar. A escolha do vértice a visitar será sucessivamente o que possua, no momento da seleção, a menor distância à origem e ainda não tenha sido visitado.
 - Algoritmo *Bellman-Ford* (**programação dinâmica**): aplica-se a grafos sem ciclos com arestas negativas com soma dos pesos negativa
 - Tem como base o pressuposto que havendo $|V|$ vértices, o caminho mais longo terá $|V| - 1$ arestas;
 - Deste modo, se se executar o relaxamento de todas as arestas até $|V| - 1$ vezes atingir-se-á as distâncias mínimas desde a origem para cada vértice. Se após 1 iteração completa não houver alteração de distâncias, a execução pode terminar.
 - Se após $|V|$ iterações ainda houver alteração de distâncias, tal significa que existe pelo menos um ciclo com soma das arestas negativa e, por isso, o algoritmo BF não pode ser aplicado.

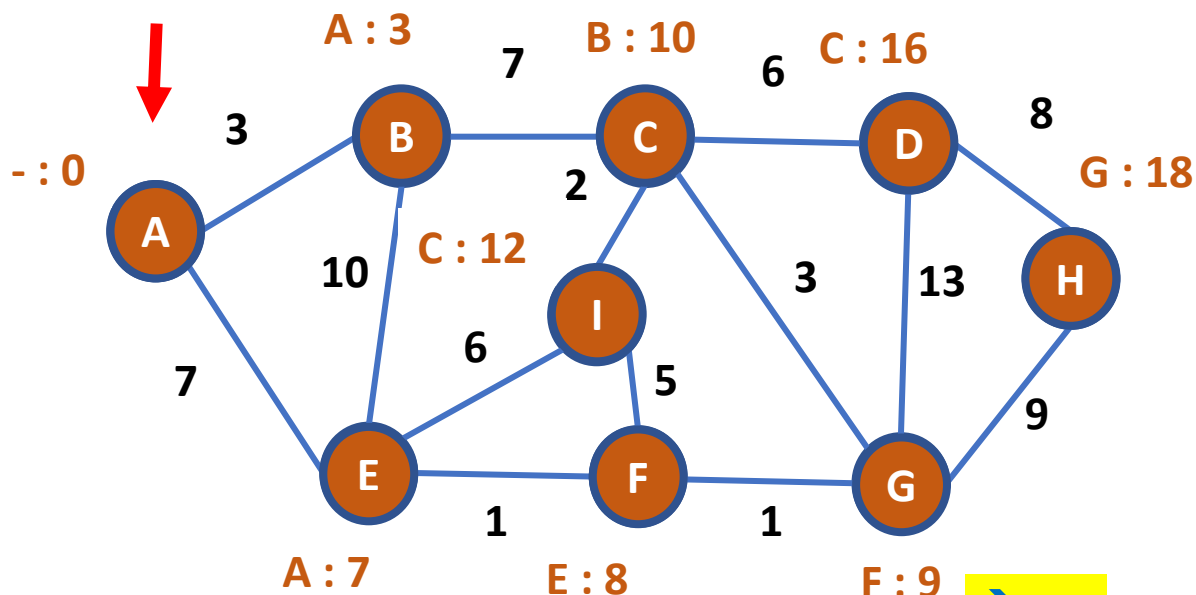
Caminho mais curto

Relaxamento (u,v)
 if ($d[u] + w(u,v) < d[v]$)
 $d[v] = d[u] + w(u,v)$

$\pi : d$
 Nó antecessor :
 distância à origem

• Algoritmo de *Dijkstra*: Complexidade temporal: $O(V+E \log V)$

1. Selecionar o vértice origem a partir do qual se pretende calcular a distância para os restantes vértices. Atribuir a esse vértice a distância 0 (zero) e ∞ aos restantes vértices.
2. Fazer o Relaxamento a partir do vértice para os vértices adjacentes e considerar esse vértice visitado
3. Enquanto houver vértices não visitados, selecionar o vértice não visitado com menor distância e aplicar o passo 2.



Distância mais curta entre A e D? $\Rightarrow 16$

Visitar	A	B	C	D	E	F	G	H	I
A	0	∞	∞	∞	∞	∞	∞	∞	∞
B		3	∞	∞	7	∞	∞	∞	∞
E			10	∞	7	∞	∞	∞	∞
F			10	∞		8	∞	∞	13
G			10	∞			9	∞	13
C			10	22				18	13
I				16				18	12
D				16				18	
H								18	

Qual o caminho mais curto entre A e H?

A
E
F
G
H

$\Rightarrow$ A-E-F-G-H

Caminho mais curto

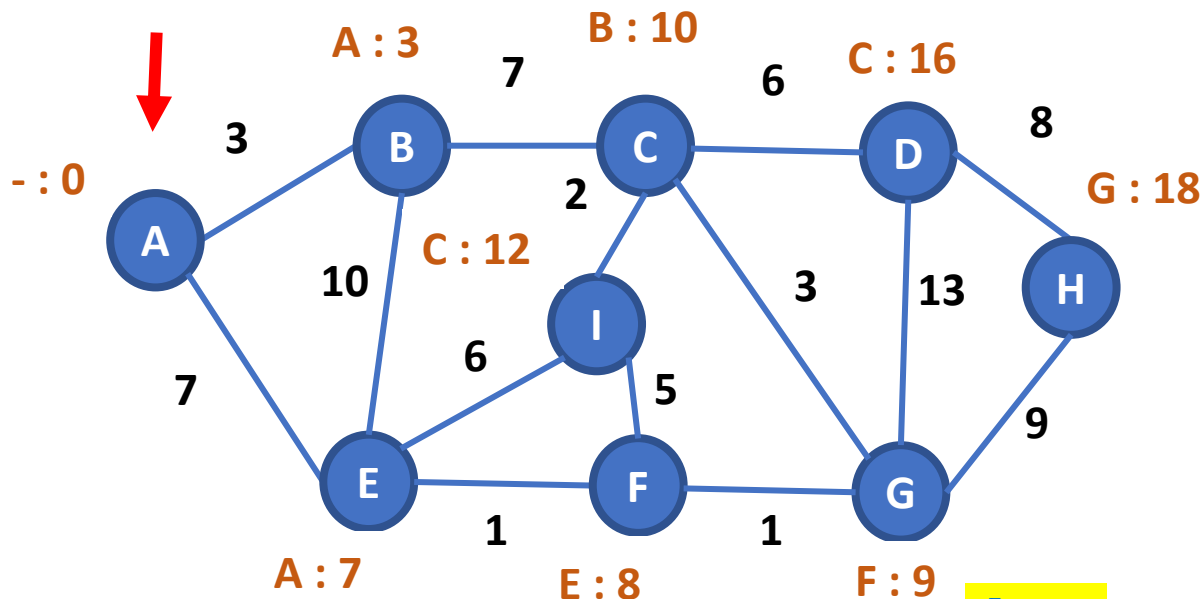
Relaxamento (u,v)
 if ($d[u] + w(u,v) < d[v]$)
 $d[v] = d[u] + w(u,v)$

$\pi : d$
 Nó antecessor :
 distância à origem

• Algoritmo de *Bellman-Ford*:

1. Selecionar o vértice origem a partir do qual se pretende calcular a distância para os restantes vértices. Atribuir a esse vértice a distância 0 (zero) e ∞ aos restantes vértices.
2. Fazer o Relaxamento de todas as arestas
3. Se o número de iterações for inferior a $|V|$ e 1 iteração completa produzir alterações de distâncias, ir para o passo 2. Senão terminar.

Complexidade temporal: $O(V \log V)$



Distância mais curta entre A e D?

→ 16

Iteração/ Aresta	1ª	2ª	3ª	4ª	5ª	6ª	7ª	8ª
(A,B)	✓	✓						
(A,E)	✓	✓						
(B,C)	✓	✓						
(C,D)	✓	✓						
(E,F)	✓	✓						
(E,I)	✓	✓						
(E,B)	✓	✓						
(F,I)	✓	✓						
(C,I)	✓	✓						
(C,G)	✓	✓						
(F,G)	✓	✓						
(D,G)	✓	✓						
(D,H)	✓	✓						
(G,H)	✓	✓						

Qual o caminho mais curto entre A e H?

A
E
F
G
H

→ A-E-F-G-H

Caminho mais curto

Relaxamento (u,v)
if ($d[u] + w(u,v) < d[v]$)
 $d[v] = d[u] + w(u,v)$

- Caminho mais curto de todos os vértices do grafo entre si (*all pairs shortest path*).
 - Algoritmo de *Floyd-Warshall* (**programação dinâmica**):
 - Calcular para cada um dos $|V|$ vértices, por via do relaxamento, o custo do caminho que permita obter a menor distância para cada um dos outros vértices.
 - Solução alternativa é a de aplicar $|V|$ vezes um dos algoritmos *single source shortest path*:
 - Algoritmo de *Dijkstra*
 - Algoritmo *Bellman-Ford*

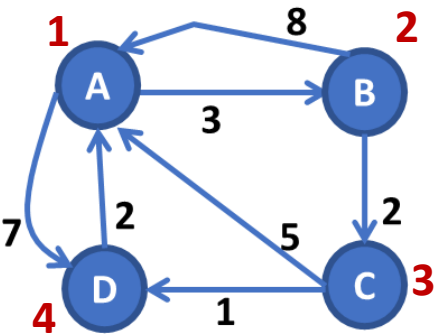
Caminho mais curto

$$D^k[i,j] = \min \{D^{k-1}[i,j], D^{k-1}[i,k] + D^{k-1}[k,j]\}$$

• Algoritmo *Floyd-Warshall*

Complexidade temporal: $O(V^3)$

- Consiste em gerar sucessivas matrizes $D^1, D^2, \dots, D^k$ com $k = |V|$. A matriz D^k contém, em cada posição a informação sobre o custo mínimo de ir de um qualquer vértice (linha de D^k) para qualquer outro vértice (coluna de D^k) passando pelo vértice k .
- O termo geral $D^k[i,j]$ de cada uma das matrizes $D^1, D^2, \dots, D^k$ é obtido por programação dinâmica.



$$D^0 = \begin{array}{c|cccc} & A & B & C & D \\ \hline A & 0 & 3 & \infty & 7 \\ B & 8 & 0 & 2 & \infty \\ C & 5 & \infty & 0 & 1 \\ D & 2 & \infty & \infty & 0 \end{array}$$

$$D^1 = \begin{array}{c|cccc} & A & B & C & D \\ \hline A & 0 & 3 & \infty & 7 \\ B & 8 & 0 & 2 & 15 \\ C & 5 & 8 & 0 & 1 \\ D & 2 & 5 & \infty & 0 \end{array}$$

$D^0[2,3] = 2$
 $D^0[2,1] + D^0[1,3] = 8 + \infty$
 $2 < 8 + \infty$

$D^0[2,4] = \infty$
 $D^0[2,1] + D^0[1,4] = 8 + 7$
 $\infty > 8 + 7$

$$D^2 = \begin{array}{c|cccc} & A & B & C & D \\ \hline A & 0 & 3 & 5 & 7 \\ B & 8 & 0 & 2 & 15 \\ C & 5 & 8 & 0 & 1 \\ D & 2 & 5 & 7 & 0 \end{array}$$

$D^1[1,3] = \infty$
 $D^1[1,2] + D^1[2,3] = 3 + 2$
 $\infty > 3 + 2$

$D^1[1,4] = 7$
 $D^1[1,2] + D^1[2,4] = 3 + 15$
 $7 < 3 + 15$

$$D^3 = \begin{array}{c|cccc} & A & B & C & D \\ \hline A & 0 & 3 & 5 & 6 \\ B & 7 & 0 & 2 & 3 \\ C & 5 & 8 & 0 & 1 \\ D & 2 & 5 & 7 & 0 \end{array}$$

$$D^2[1,2] = 3$$

 $D^2[1,3] + D^2[3,2] = 5 + 8$
 $3 < 5 + 8$

$$D^4 = \begin{array}{c|cccc} & A & B & C & D \\ \hline A & 0 & 3 & 5 & 6 \\ B & 5 & 0 & 2 & 3 \\ C & 3 & 6 & 0 & 1 \\ D & 2 & 5 & 7 & 0 \end{array}$$

```
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            D[i, j] = Math.Min(D[i, j], D[i, k] + D[k, j]);
        }
    }
}
```

Objetivos de aprendizagem

- Fornecer conhecimentos para compreender:
 - Estruturas de dados e algoritmos baseados em grafos

Referências

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3-rd edition). MIT Press and McGraw-Hill.
- Syed Mohiuddin, Data Structures and Algorithms: In Depth using C#, <https://www.udemy.com/>

Anexos

Grafos

- Implementação de Grafo

```
class Graph {  
    private int vertices;  
    private int[,] adjacencyMatrix;  
    public Graph(int n) {  
        vertices = n;  
        adjacencyMatrix = new int[n, n];  
    }  
    public void InsertEdge(int u, int v, int w) {  
        adjacencyMatrix[u, v] = w;  
    }  
    public void RemoveEdge(int u, int v) {  
        adjacencyMatrix[u, v] = 0;  
    }  
    public bool ExistEdge(int u, int v) {  
        return adjacencyMatrix[u, v] != 0;  
    }  
    public int vertexCount() {  
        return vertices;  
    }  
    public int EdgeCount() {  
        int count = 0;  
        for (int i = 0; i < vertices; i++) {  
            for (int j = 0; j < vertices; j++) {  
                if (adjacencyMatrix[i, j] != 0)  
                    count++;  
            }  
        }  
        return count;  
    }  
}
```

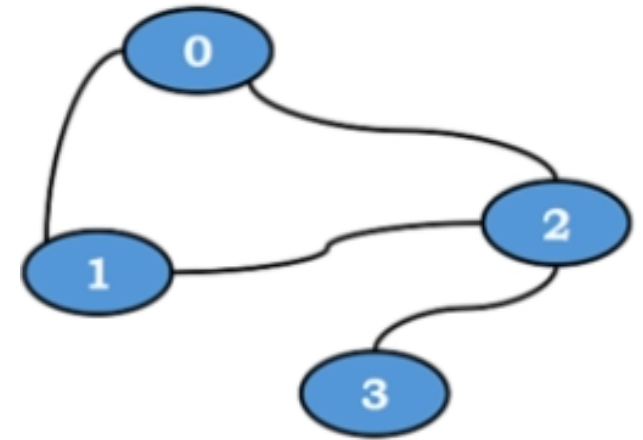
```
    public void Edges() {  
        for (int i = 0; i < vertices; i++) {  
            for (int j = 0; j < vertices; j++) {  
                if (adjacencyMatrix[i, j] != 0)  
                    Console.WriteLine(i + "," + j);  
            }  
        }  
    }  
    public int Outdegree(int v) {  
        int count = 0;  
        for (int j = 0; j < vertices; j++) {  
            if (adjacencyMatrix[v, j] != 0)  
                count++;  
        }  
        return count;  
    }  
    public int Indegree(int u) {  
        int count = 0;  
        for (int i = 0; i < vertices; i++) {  
            if (adjacencyMatrix[i, u] != 0)  
                count++;  
        }  
        return count;  
    }  
    public void Display() {  
        for (int i = 0; i < vertices; i++) {  
            for (int j = 0; j < vertices; j++) {  
                Console.Write(adjacencyMatrix[i, j] + "\t ");  
            }  
            Console.WriteLine();  
        }  
    }  
}
```

Grafos

- Instanciação de grafo não-dirigido

```
static void Main(string[] args) {  
    Graph g = new Graph(4);  
    Console.WriteLine("Adjancency Matrix");  
    g.InsertEdge(0, 1, 1);  
    g.InsertEdge(0, 2, 1);  
    g.InsertEdge(1, 0, 1);  
    g.InsertEdge(1, 2, 1);  
    g.InsertEdge(2, 0, 1);  
    g.InsertEdge(2, 1, 1);  
    g.InsertEdge(2, 3, 1);  
    g.InsertEdge(3, 2, 1);  
    g.Display();  
    Console.WriteLine("Vertices count: " + g.VertexCount());  
    Console.WriteLine("Edges count: " + g.EdgeCount());  
    g.Edges();  
    Console.WriteLine("Edge (1,3)? " + g.ExistEdge(1, 3));  
    Console.WriteLine("Edge (1,2)? " + g.ExistEdge(1, 2));  
    Console.WriteLine("Indegree Vertex 2? " + g.Indegree(2));  
    Console.WriteLine("Remove Edge (1,2)");  
    g.RemoveEdge(1, 2);  
    Console.WriteLine("Edge (1,2)? " + g.ExistEdge(1, 2));  
}
```

```
Adjancency Matrix  
0      1      1      0  
1      0      1      0  
1      1      0      1  
0      0      1      0  
Vertices count: 4  
Edges count: 8  
0,1  
0,2  
1,0  
1,2  
2,0  
2,1  
2,3  
3,2  
Edge (1,3)? False  
Edge (1,2)? True  
Indegree Vertex 2? 3  
Remove Edge (1,2)  
Edge (1,2)? False
```



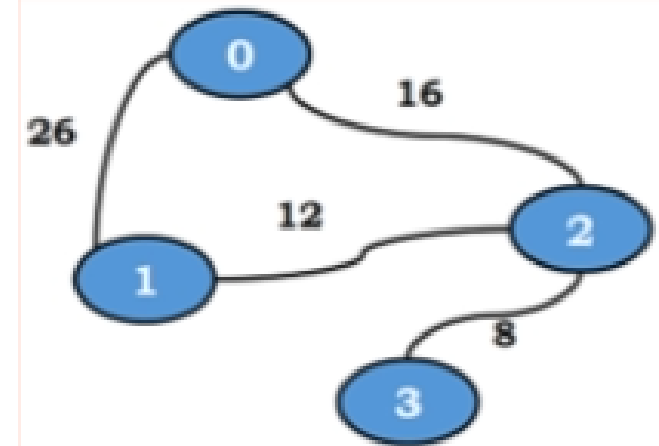
Undirected Graph

Grafos

- Instanciação de grafo pesado não-dirigido

```
static void Main(string[] args) {  
    Graph g = new Graph(4);  
    Console.WriteLine("Adjancency Matrix");  
    g.InsertEdge(0, 1, 26);  
    g.InsertEdge(0, 2, 16);  
    g.InsertEdge(1, 0, 26);  
    g.InsertEdge(1, 2, 12);  
    g.InsertEdge(2, 0, 16);  
    g.InsertEdge(2, 1, 12);  
    g.InsertEdge(2, 3, 8);  
    g.InsertEdge(3, 2, 8);  
    g.Display();  
    Console.WriteLine("Vertices count: " + g.VertexCount());  
    Console.WriteLine("Edges count: " + g.EdgeCount());  
    g.Edges();  
    Console.WriteLine("Edge (1,3)? " + g.ExistEdge(1, 3));  
    Console.WriteLine("Edge (1,2)? " + g.ExistEdge(1, 2));  
    Console.WriteLine("Indegree Vertex 2? " + g.Indegree(2));  
    Console.WriteLine("Remove Edge (1,2)");  
    g.RemoveEdge(1, 2);  
    Console.WriteLine("Edge (1,2)? " + g.ExistEdge(1, 2));  
}
```

```
Adjancency Matrix  
0      26      16      0  
26      0      12      0  
16      12      0      8  
0      0      8      0  
Vertices count: 4  
Edges count: 8  
0,1  
0,2  
1,0  
1,2  
2,0  
2,1  
2,3  
3,2  
Edge (1,3)? False  
Edge (1,2)? True  
Indegree Vertex 2? 3  
Remove Edge (1,2)  
Edge (1,2)? False
```

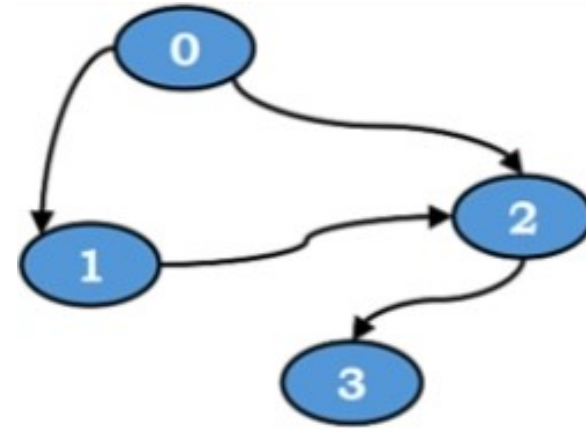


Grafos

- Instanciação de grafo dirigido

```
static void Main(string[] args) {  
    Graph g = new Graph(4);  
    Console.WriteLine("Adjancency Matrix");  
    g.InsertEdge(0, 1, 1);  
    g.InsertEdge(0, 2, 1);  
    g.InsertEdge(1, 2, 1);  
    g.InsertEdge(2, 3, 1);  
    g.Display();  
    Console.WriteLine("Vertices count: " + g.VertexCount());  
    Console.WriteLine("Edges count: " + g.EdgeCount());  
    g.Edges();  
    Console.WriteLine("Edge (1,3)? " + g.ExistEdge(1, 3));  
    Console.WriteLine("Edge (1,2)? " + g.ExistEdge(1, 2));  
    Console.WriteLine("Edge (2,1)? " + g.ExistEdge(2, 1));  
    Console.WriteLine("degree Vertex 2? " + (g.Indegree(2)+g.Outdegree(2)));  
    Console.WriteLine("Indegree Vertex 2? " + g.Indegree(2));  
    Console.WriteLine("Outdegree Vertex 2? " + g.Outdegree(2));  
    Console.WriteLine("Remove Edge (1,2)");  
    g.RemoveEdge(1, 2);  
    Console.WriteLine("Edge (1,2)? " + g.ExistEdge(1, 2));  
}
```

```
Adjancency Matrix  
0      1      1      0  
0      0      1      0  
0      0      0      1  
0      0      0      0  
Vertices count: 4  
Edges count: 4  
0,1  
0,2  
1,2  
2,3  
Edge (1,3)? False  
Edge (1,2)? True  
Edge (2,1)? False  
degree Vertex 2? 3  
Indegree Vertex 2? 2  
Outdegree Vertex 2? 1  
Remove Edge (1,2)  
Edge (1,2)? False
```

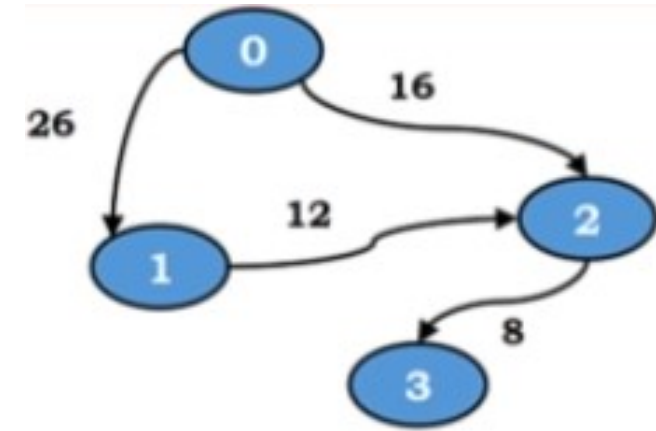


Grafos

- Instanciação de grafo dirigido pesado

```
static void Main(string[] args) {  
    Graph g = new Graph(4);  
    Console.WriteLine("Adjancency Matrix");  
    g.InsertEdge(0, 1, 26);  
    g.InsertEdge(0, 2, 16);  
    g.InsertEdge(1, 2, 12);  
    g.InsertEdge(2, 3, 8);  
    g.Display();  
    Console.WriteLine("Vertices count: " + g.VertexCount());  
    Console.WriteLine("Edges count: " + g.EdgeCount());  
    g.Edges();  
    Console.WriteLine("Edge (1,3)? " + g.ExistEdge(1, 3));  
    Console.WriteLine("Edge (1,2)? " + g.ExistEdge(1, 2));  
    Console.WriteLine("Edge (2,1)? " + g.ExistEdge(2, 1));  
    Console.WriteLine("degree Vertex 2? " + (g.Indegree(2)+g.Outdegree(2)));  
    Console.WriteLine("Indegree Vertex 2? " + g.Indegree(2));  
    Console.WriteLine("Outdegree Vertex 2? " + g.Outdegree(2));  
    Console.WriteLine("Remove Edge (1,2)");  
    g.RemoveEdge(1, 2);  
    Console.WriteLine("Edge (1,2)? " + g.ExistEdge(1, 2));  
}
```

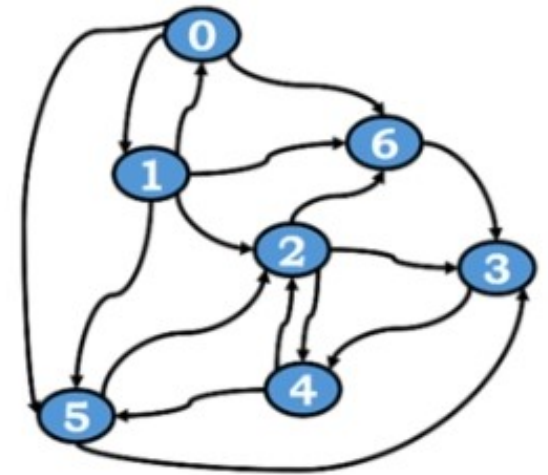
```
Adjancency Matrix  
0      26      16      0  
0       0      12      0  
0       0       0      8  
0       0       0      0  
Vertices count: 4  
Edges count: 4  
0,1  
0,2  
1,2  
2,3  
Edge (1,3)? False  
Edge (1,2)? True  
Edge (2,1)? False  
degree Vertex 2? 3  
Indegree Vertex 2? 2  
Outdegree Vertex 2? 1  
Remove Edge (1,2)  
Edge (1,2)? False
```



Procura em Largura (*Breadth-First Search*)

```
public void BFS(int s) {
    int i = s;
    QueueLinked q = new QueueLinked();
    int[] visited = new int[vertices];
    Console.Write(i + " ");
    visited[i] = 1;
    q.Enqueue(i);
    while (!q.IsEmpty()) {
        i = q.Dequeue();
        for (int j = 0; j < vertices; j++) {
            if (adjacencyMatrix[i, j] == 1 && visited[j] == 0) {
                Console.Write(j + " ");
                visited[j] = 1;
                q.Enqueue(j);
            }
        }
    }
}
```

```
static void Main(string[] args) {
    Graph g = new Graph(7);
    g.InsertEdge(0, 1, 1);
    g.InsertEdge(0, 5, 1);
    g.InsertEdge(0, 6, 1);
    g.InsertEdge(1, 0, 1);
    g.InsertEdge(1, 5, 1);
    g.InsertEdge(1, 2, 1);
    g.InsertEdge(1, 6, 1);
    g.InsertEdge(2, 3, 1);
    g.InsertEdge(2, 4, 1);
    g.InsertEdge(2, 6, 1);
    g.InsertEdge(3, 4, 1);
    g.InsertEdge(4, 2, 1);
    g.InsertEdge(4, 5, 1);
    g.InsertEdge(5, 2, 1);
    g.InsertEdge(5, 3, 1);
    g.InsertEdge(6, 3, 1);
    Console.WriteLine("Breadth First Search ");
    g.BFS(0);
    Console.WriteLine();
}
```



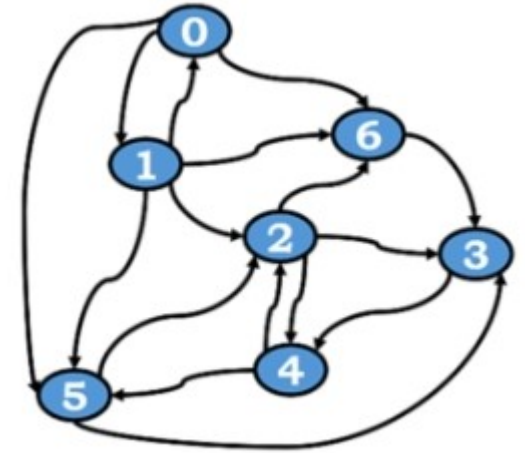
Breadth First Search
0 1 5 6 2 3 4

Procura em Profundidade (*Depth-First Search*)

```
public Graph(int n) {  
    vertices = n;  
    adjacencyMatrix = new int[n, n];  
    visited = new int[vertices];  
}
```

```
int[] visited;  
public void DFS(int s) {  
    if (visited[s] == 0) {  
        Console.Write(s + " ");  
        visited[s] = 1;  
        for (int j = 0; j < vertices; j++) {  
            if (adjacencyMatrix[s, j] == 1 && visited[j] == 0) {  
                DFS(j);  
            }  
        }  
    }  
}
```

```
static void Main(string[] args) {  
    Graph g = new Graph(7);  
    g.InsertEdge(0, 1, 1);  
    g.InsertEdge(0, 5, 1);  
    g.InsertEdge(0, 6, 1);  
    g.InsertEdge(1, 0, 1);  
    g.InsertEdge(1, 5, 1);  
    g.InsertEdge(1, 2, 1);  
    g.InsertEdge(1, 6, 1);  
    g.InsertEdge(2, 3, 1);  
    g.InsertEdge(2, 4, 1);  
    g.InsertEdge(2, 6, 1);  
    g.InsertEdge(3, 4, 1);  
    g.InsertEdge(4, 2, 1);  
    g.InsertEdge(4, 5, 1);  
    g.InsertEdge(5, 2, 1);  
    g.InsertEdge(5, 3, 1);  
    g.InsertEdge(6, 3, 1);  
    Console.WriteLine("Depth First Search ");  
    g.DFS(0);  
    Console.WriteLine();  
}
```



```
Depth First Search  
0 1 2 3 4 5 6
```